

Previsão IBOVESPA

Victoria Costi Menezes

O Projeto

Foi desenvolvido um modelo preditivo **classificador** com uma **acurácia > 75%** para prever se o índice IBOVESPA irá fechar em alta ou baixa com base nos dados disponíveis no momento de abertura.

Utilizamos exclusivamente **dados do próprio IBOVESPA** para treinar e testar o modelo, construindo features adicionais quando necessário. Nossos dados brutos vão de **Janeiro/2010** até **Junho/2025**. Todos os testes serão performados no **último mês útil**.

O repositório do projeto pode ser encontrado aqui:

github.com/victoria-menezes/fiap-data-analytics-TC2

O modelo final escolhido foi um **modelo classificador**, e portanto será nesse notebook que esse relatório irá focar. O notebook de Time Series permanecerá exposto no repositório para registrar o estudo feito.

Fonte dos dados: br.investing.com/indices/bovespa-historical-data

Os dados

Pela **figura abaixo** observamos a linha do tempo do valor de fechamento junto de sua média móvel. Por esse gráfico o impacto da COVID-19 ficou nítido, e em nossos testes observamos que a retirada desse período de Jan/2020 - Jan/2021 melhorou a acurácia de nossos modelos, portanto ele não foi incluso no treinamento final.



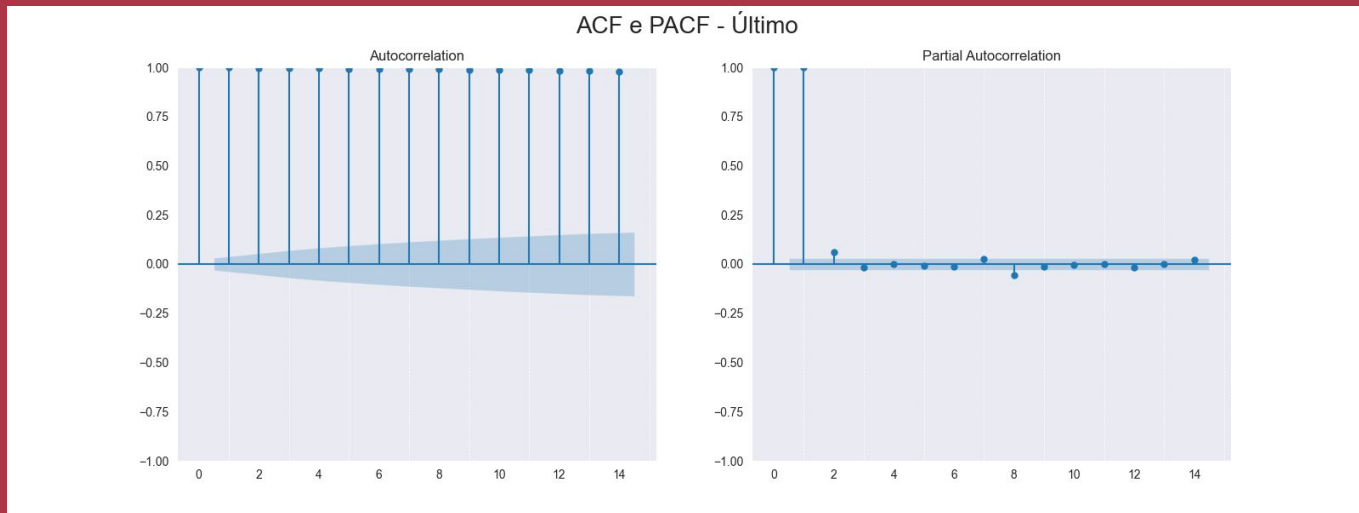
Os dados

Pelo zoom da figura anterior observamos como o valor se comporta no dia-a-dia. A variação diária é volátil, tendo picos e quedas repentinos mesmo quando o valor bruto segue uma tendência (mais facilmente vista pela média móvel), o valor pode subir ou descer de uma maneira errática. É essa subida e descida que os modelos foram construídos para prever.



Os dados

Pelo ACF e PACF, confirmamos que o valor de fechamento segue uma tendência — visível pelo padrão do ACF — e que não apresenta forte autocorrelação parcial em períodos além de 1. Se há uma sazonalidade simples nos dados, ela não é significativa o suficiente para se manifestar nessas figuras. Os dados então são uma **série temporal não estacionária, sem sazonalidade**.



Preparação

Os modelos candidatos foram **classificadores**, ou seja, não tinham uma inerente capacidade de interpretar o aspecto temporal dos dados. Sendo assim, tivemos que re-inserir isso por meio de features.

Além disso, precisamos considerar a realidade: dos dados iniciais — fechamento, máxima, mínima — apenas temos acesso à abertura do dia na hora de prever. Assim, **esses features, e os outros que construímos, se referem ao dia anterior**. A única exceção é a **abertura do dia, e os features que se referem à data**.

Os dados foram escalonados através do StandardScaler antes de serem usados para o treino.

data	close_TBD	target	delta	return	target_lag1	target_lag2	target_lag3	target_lag4	target_lag5	target_lag6	...	rsi_5	rsi_10	rsi_20	rsi_22	rsi_66	ma5	ma22	ma66	ma132	ma252
2025-06-26	137.114	1.0	-1.398	-0.010192	0.0	1.0	0.0	0.0	0.0	0.0	...	30.875893	40.244116	48.292885	49.179419	53.088499	137.0632	137.431227	134.521394	129.366598	129.646520
2025-06-27	136.866	0.0	1.347	0.009921	1.0	0.0	1.0	0.0	0.0	0.0	...	49.871249	49.450843	52.342817	52.777307	54.115465	136.7426	137.384773	134.599561	129.433917	129.709111
2025-06-30	138.855	1.0	-0.248	-0.001809	0.0	1.0	0.0	1.0	0.0	0.0	...	46.904849	47.939849	51.560159	52.066335	53.894899	136.6926	137.263182	134.668061	129.489015	129.765575

Preparação

- **delta:** A diferença do fechamento de ontem, com o de anteontem;
 - **return:** A diferença percentual do o fechamento de ontem com o de anteontem;
 - **target_lag[n]:** O target de n dias atrás;
 - n = 1 a 10
 - **delta_lag[n]:** O delta de n dias atrás
 - n = 1 a 10
 - **high:** A máxima de ontem;
 - **low:** A mínima de ontem;
 - **volatility:** Diferença entre high e low
 - **volatility_relative:** Diferença entre high e low relativa à low
 - **open:** Valor de abertura
 - **rsi_[n]:** Indicador RSI de ontem, abrangendo n dias;
 - n = 3, 5, 10, 20, 22, 66
 - **ma[n]:** A média móvel de ontem, até n dias antes;
 - n = 5, 22, 66, 132, 252
 - **weekday:** Dia da semana, de 0 a 6, com 0 sendo segunda-feira;
 - **month:** Mês numérico;
 - **month_day:** Dia do mês;
-

Preparação

Como estamos lidando com dados **temporais**, a separação de **treino e teste** foi feita por períodos. **Um mês útil** foi recortado do final do dataset para ser utilizado como teste. Além disso, foi definido uma instância `TimeSeriesSplit` com 4 'folds' para fazer a Cross-Validation.

```
kf_params = {  
    'splits': 4,  
    'shuffle': True,  
    'random_state': SEED  
}  
  
kft = TimeSeriesSplit(  
    n_splits = kf_params['splits'],  
    test_size = n_predictions  
)
```


Escolha de modelo

Os seguintes modelos foram selecionados para o nosso estudo:

- **LogisticRegression**, um modelo que tenta determinar a probabilidade de um resultado;
- **XGBoost**, um modelo que constrói árvores por várias iterações;
- **RandomForestClassifier**, um modelo que constrói várias árvores e se baseia nas decisões da 'floresta';
- **K-Nearest Neighbors (KNN)**, um modelo que identifica os dados vizinhos para realizar sua predição;
- **Support Vector Classifier (SVC)**, um modelo que tenta traçar hiperplanos entre os dados para classificá-los em grupos;

São todos modelos classificadores que terão como output **0 ou 1** como respostas para a pergunta: **a bolsa irá fechar em alta?**

Seus resultados foram então comparados com um modelo **baseline**, que apenas repete o resultado do dia anterior. O objetivo inicial é **superar o modelo baseline**, e o objetivo final é **atingir ou superar a acurácia de 75%**.

Escolha de modelo

Através do GridSearch, (ou RandomSearch, dependendo do modelo), construímos uma maneira de procurar os melhores hiperparâmetros dentro de opções já definidas. Utilizamos Pipelines para esse processo, e fornecemos grids de possibilidades no notebook.

```
pipe_forest = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', RandomForestClassifier())
])

param_grid_forest = {
    'clf__n_estimators': [5, 10, 20, 30],
    'clf__max_depth': [7, 10, 15, 25, 30],
    'clf__random_state': [SEED]
}

grid_forest = GridSearchCV(
    estimator = pipe_forest,
    param_grid = param_grid_forest,
    cv = kft)

grid_forest.fit(X_train,
                y_train)
```

Baseline - 50.0%



KNN - 54.4%



LogisticRegression - 54.4%



XGBoost- 59.1%

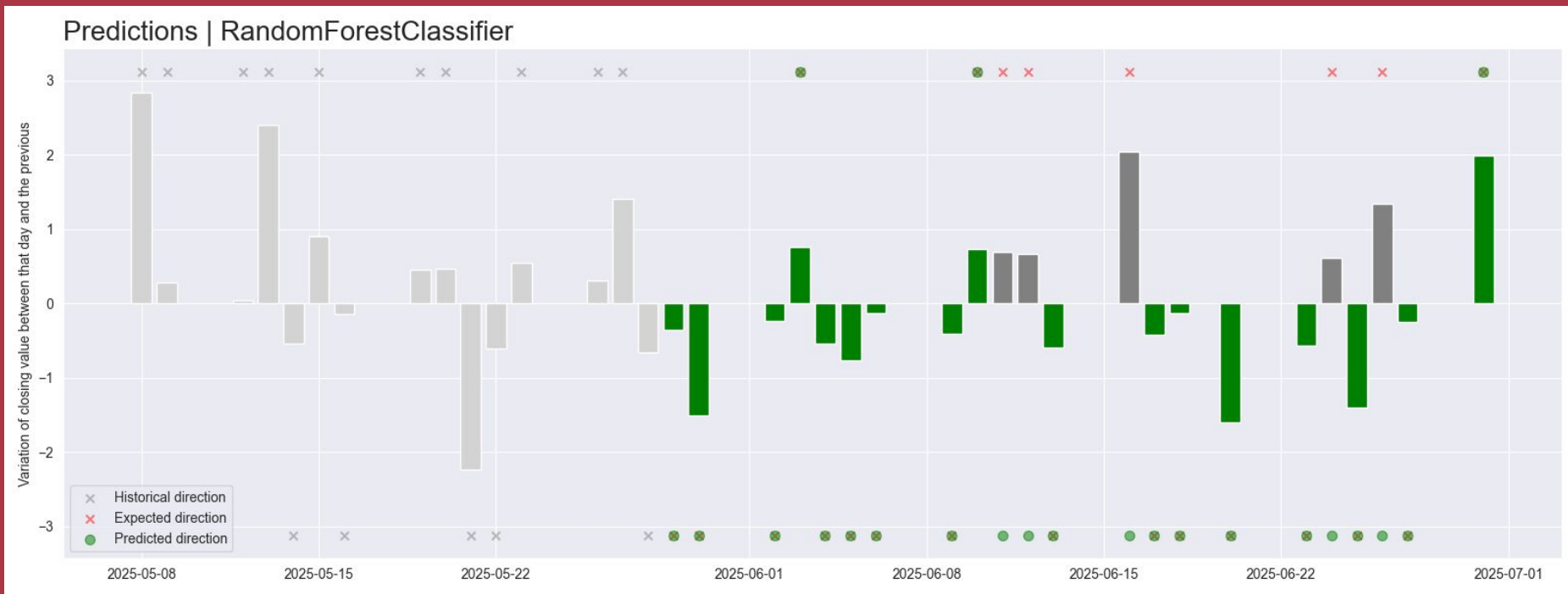


SVC - 68.2%

Predictions | SVM



RandomForestClassifier - 77.3%



Escolha de modelo

Todos os modelos performaram melhor que o Baseline em acurácia.

O modelo final escolhido pelo critério de >75% de acurácia foi o **RandomForestClassifier**, com o modelo SVC também se destacando com 68% de acurácia. O RFC se destacou principalmente na sua precisão, não retornando **nenhum falso positivo**.

O modelo aparenta favorecer resultados negativos, resultando assim em um recall mais baixo. Essa é uma possível área de melhora, mas se demonstrou uma excelente opção para um aprofundamento.

	accuracy	precision	recall
model			
RandomForestClassifier	0.772727	1.000000	0.375
SVM	0.681818	0.571429	0.500
XGBoost	0.590909	0.333333	0.125
LogisticRegression	0.545455	0.400000	0.500
KNN	0.545455	0.250000	0.125
Baseline	0.500000	0.285714	0.250

```
model = RandomForestClassifier(  
    max_depth = 7,  
    n_estimators = 5,  
    random_state = SEED  
)
```

SEED utilizada foi 152